

Website **workbook** and next-stage roadmap

A practical view of what the game-night website can do today, how the new backend works, what remains before launch, and how to make the visual experience more distinctively Islington.

Prepared 23 September 2026 | Current stage: local MVP

1. What exists now

Current status: The frontend and first backend milestone are working locally. The site is not yet publicly hosted, and it is not a complete booking or payment system.

Customer-facing website	Working backend
• Premium "From court to calm" campaign page • Boxwood, Islington and JackJumpers story • Game-night choice area • Offer and package inclusions • Responsive phone, tablet and desktop layout	• ASP.NET Core Minimal API in C# • SQLite database created automatically • Enquiry validation on the server • 15-second duplicate-submission cooldown • Health endpoint for deployment checks

2. How the website works

Step	Guest experience	System behaviour
1. Discover	Guest reads the narrative, sees the package and explores Islington.	Static HTML, CSS and JavaScript load in the browser.
2. Choose a night	Guest selects the next home game or a future home game.	The selected option is included in the enquiry request.
3. Enquire	Guest enters name, email and an optional note.	Browser validation runs first; the API validates again.
4. Save	Guest receives a confirmation message.	API writes the enquiry to SQLite and returns a confirmation ID.
5. Follow-up	Boxwood confirms availability, ticket allocation and final price.	Currently manual; no email, CRM or payment integration exists yet.

3. Work still required before launch

Priority 1 - Hosting and operations

- Host frontend over HTTPS.
- Host API separately or in the same deployment.
- Use managed PostgreSQL or hosted SQLite-compatible storage for production.
- Set backups, logs, uptime monitoring and a real domain.

Priority 2 - Booking workflow

- Add staff-only enquiry dashboard.
- Send email notifications to Boxwood.
- Track enquiry status: new, contacted, confirmed, cancelled.
- Connect real game dates, room inventory and ticket availability.

Priority 3 - Commercial flow

- Confirm package tiers and pricing.
- Add guest count, stay date and room preference fields.
- Add payment or deposit provider after availability is confirmed.
- Add terms, cancellation policy and privacy notice.

Priority 4 - Quality and accessibility

- Add automated API and browser tests.
- Test keyboard navigation, focus states and screen readers.
- Compress and serve responsive image sizes.
- Measure conversion, performance and form completion.

4. Hosting options

Simple recommended setup

- Frontend: Netlify, Cloudflare Pages or GitHub Pages.
- API: Render, Railway, Fly.io or Azure App Service running ASP.NET Core.
- Database: managed PostgreSQL for production; keep SQLite for local development only.
- This split keeps the static site fast while the API handles enquiries.

Single-provider option

- Deploy frontend and API to Azure App Service, with Azure Database for PostgreSQL and Application Insights.
- This is more operationally coherent but usually costs more and needs more setup.

Not yet hosted: The current GitHub Pages workflow is suitable for the static frontend, but it cannot run the C# API or persist production enquiries by itself.

5. Current security exposure

The current build is an MVP and should not be treated as a production booking system yet.

HIGH	Open CORS policy. The API allows requests from any origin. Restrict this to the real website domain before hosting.
HIGH	No staff authentication or authorisation. There is no protected admin area, so staff data access controls must be added.
MEDIUM	In-memory rate limiting. It resets on restart and does not coordinate across multiple servers.
MEDIUM	SQLite operational risk. A local file is not ideal for concurrent production traffic, backups or multi-instance hosting.
MEDIUM	Forwarded IP trust. Configure trusted proxy handling so clients cannot spoof the rate-limit identity.
LOW	Privacy and retention are undefined. Add a privacy notice, retention period, access controls, deletion process and encrypted production storage.

Already improved: SQL parameters are used, server-side input validation exists, no secrets are hardcoded, and the database file is ignored by Git.

6. UI design and image direction

UI additions

- Real game-date cards with opponent, tip-off and availability.
- Package tier comparison: Essential, Premium and Fireside.
- Guest count, date and room preference fields.
- Sticky mobile enquiry CTA.
- Confirmation screen with next steps and contact details.
- Accessible error, loading and success states.

More unique Islington look

- Replace generic stock fireplace imagery with approved Islington photography.
- Add a visual house tour: arrival, room, Emerald Lounge, fireplace and morning light.
- Use close crops of timber, books, glassware, linen and heritage details.
- Pair warm interior images with one Hobart/Mount Wellington context image.
- Use a consistent warm film treatment.

Current image situation: The page uses an external Islington logo and one Unsplash fireplace image. The next design pass should replace these with licensed, brand-approved Islington assets and responsive local copies for reliability and performance.

7. Recommended delivery sequence

- Connect real data: game schedule, ticket allocation, package inventory and enquiry statuses.
- Harden the API: HTTPS, restricted CORS, production database, distributed rate limiting, authentication and logging.
- Deploy a staging environment with non-production data.
- Refresh the visual system with approved Islington photography, package tiers, dates and conversion-focused CTA.
- Launch carefully with privacy terms, monitoring, backups, staff process and a tested enquiry-to-booking handoff.

Prepared from the local Boxwood x Islington x JackJumpers project at its local MVP stage. This document is a planning reference, not a security certification or production readiness approval.